

Aplicación Móvil - React Native + Express

Pokedex (USUARIOS - ENCICLOPEDIA - MOVIMIENTOS - EQUIPOS)



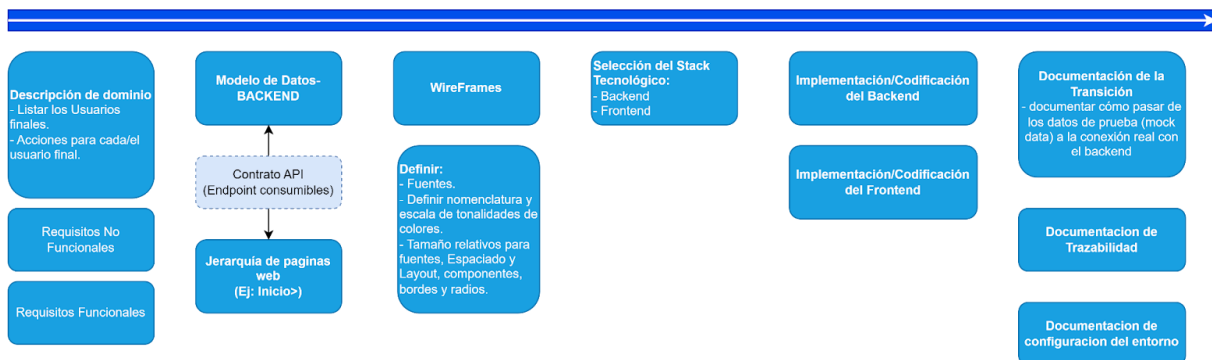
Introducción

Esta aplicación móvil consta con características deseadas para el aprendizaje desde operaciones como lo son crear, leer, actualizar y eliminar hasta además temas que son comunes en desarrollo como la gestión de usuarios, este ultimo el tema se expande desde la autenticación, validación y rutas protegidas que solo serían aptas para dicho usuario con el fin de proporcionar seguridad.

Ejecución

Desarrollar una aplicación desde cero es un reto si uno no sabe por dónde comenzar. Si de entrada comenzamos aleatoriamente, el proyecto tenderá a generar escenarios que implican una deuda técnica. Por eso mismo, he decidido crear una plantilla base la cual consta de pasos a seguir no sólo prácticos sino teóricos que ayude al usuario a seguir un hilo de trabajo con el fin de crear un producto final mantenible, escalable y sobre todo entendible para el resto de personas que van a trabajar en adelante. Estos pasos son visualizables en la **Figura N.º 1**.

Figura N.º 1



Nota. Imagen a detalle en [Draw.io](https://draw.io)

Descripción

En laboratorio de programación 2023, se desarrolló un sitio web relacionado con la serie animada Pokémon, más específico con una "Pokedex". En esta nuevo proyecto, el objetivo es crear una sistema-app-móvil (con API propia que será consumida para satisfacer las necesidades).

Los usuarios que usaran la aplicación serán los siguientes:

- **Registrados:** Quieren ver información de los Pokémon, movimientos. Adicional a esto podrá crear su propio equipo Pokémon con las funciones esenciales(borrar, editar).

La aplicación es usada como herramienta para planificar equipos competitivos fuera del juego original.

La aplicación dispondrá de la generación [1-5]

Requisitos Funcionales (RF)

- RF01: Registro de usuario
 - Campos requeridos: nombre de usuario, email, contraseña.
 - Validar que email y usuario sean únicos.
 - Contraseña mínima: 6 caracteres.
- RF02: Inicio de sesión
 - Login con usuario/email y contraseña
 - Mostrar mensaje claro si credenciales son incorrectas
- RF03: Consulta de Pokémon
 - Listar todos los Pokémon (generación [1-5]) con imagen, nombre, tipos, id.
 - Búsqueda por nombre (coincidencia parcial)
 - Ver detalle de Pokémon: tipos, estadística base, movimientos, habilidades.
- RF04: Consulta de movimientos
 - Listar movimientos disponibles
 - Filtrar por tipo de movimiento (OPCIONAL)
 - Ver detalle: poder, precisión, PP, descripción.
- RF05: Creación de equipos
 - Nombre del equipo (obligatorio, único por usuario).
 - Añadir Pokémon al equipo (mínimo 1, máximo 6)
 - Guardar equipo en la base de datos.
- RF08: Gestión de equipos existentes
 - Listar todos los equipos del usuario.
 - Editar equipo existente (nombre, pokemons).
 - Eliminar equipo con confirmación.
- RF09: Perfil de usuario
 - Ver información del perfil.
 - Cambiar nombre de usuario (disponibilidad).
 - Cambiar contraseña (confirmar contraseña actual).

- RF10: Filtros básicos
 - Filtrar Pokémon por tipo (1 o 2 tipos).
 - Filtrar Pokémon por generación (1-5).
 - Ordenar Pokémon por: número, nombre, estadística específica.

Requisitos no Funcionales (RNF)

- RNF01: Código organizado
 - Separar lógica de negocio de la interfaz
 - Componentes reutilizables donde sea posible
 - Comentarios en código complejo.
- RNF02: Estructura clara
 - Separación por carpetas: components, screens, services, utils.
 - Nomenclatura consistente en inglés.
- RNF03: Mantenibilidad
 - El código debe ser fácil de modificar y extender.
- RNF04: Escalabilidad
 - La arquitectura debe permitir añadir nuevas funcionalidades.

Modelado de Datos

El modelar el dominio implica proporcionar información de tablas existentes así como relaciones que existen entre ellas. No especificaremos pero hicimos uso de índices para agilizar las consultas más frecuentes.

Especificaremos a continuación las siguientes:

- MR
- DDL
- MER

Modelo relacional:

usuarios(id_usuario (PK), nombre_usuario, contrasenia, fecha_creacion, email)

equipos(id_equipo (PK), nombre, fecha_creacion, id_usuario (FK))

naturaleza(id_naturaleza (PK), estadistica_mas, estadistica_menos, nombre)

pokemon(id_pokemon (PK), nombre, tipo_1, tipo_2, hp_base, ataque_base, ataque_especial_base, defensa_base, defensa_especial_base, velocidad_base)

equipo_pokemon(id_pokemon_equipo (PK), ev_hp, ev_ataque, ev_ataque_especial, ev_defensa, ev_defensa_especial, ev_velocidad, iv_hp, iv_ataque, iv_ataque_especial, iv_defensa, iv_defensa_especial, iv_velocidad, id_equipo (FK), id_naturaleza (FK), id_pokemon (FK), id_Habilidad (FK))

habilidades(id_habilidad (PK), nombre, descripcion)

movimiento(id_movimiento (PK), nombre, tipo, categoria, poder, pp, descripcion, precision)

pokemon_puede_tener_habilidad(id_pokemon (PK,FK), id_habilidad (PK,FK), habilidad_oculta)

pokemon_aprende_movimiento(id_pokemon (PK,FK), id_movimiento (PK,FK), nivel, metodo_aprendizaje)

pokemon_tiene_movimiento(id_pokemon_equipo (PK,FK), id_movimiento (PK, FK))

DDL:

```
CREATE TABLE usuarios (  
    id_usuario INTEGER PRIMARY KEY,  
    nombre_usuario TEXT NOT NULL,  
    contrasenia TEXT NOT NULL CHECK (length(contrasenia) >= 8),  
    fecha_creacion DATETIME DEFAULT CURRENT_TIMESTAMP,  
    email TEXT NOT NULL UNIQUE CHECK (email LIKE '%@%')  
);  
CREATE TABLE equipos(  
    id_equipo INTEGER PRIMARY KEY,  
    id_usuario INTEGER NOT NULL,  
    nombre TEXT DEFAULT 'sin nombre',  
    fecha_creacion datetime DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario) ON DELETE  
CASCADE ON UPDATE CASCADE
```

```
);
CREATE TABLE naturaleza(
    id_naturaleza INTEGER PRIMARY KEY,
    nombre TEXT NOT NULL,
    estadistica_mas TEXT ,
    estadistica_menos TEXT ,
    -- El IN ES PARA CUANDO UNA COL solo acepte unos valores.
    CHECK (estadistica_mas IN ('ataque', 'defensa', 'ataque_especial',
'defensa_especial', 'velocidad', 'ninguna')),
    CHECK (estadistica_menos IN ('ataque', 'defensa', 'ataque_especial',
'defensa_especial', 'velocidad', 'ninguna'))
);
CREATE TABLE pokemon (
    id_pokemon INTEGER PRIMARY KEY,
    nombre TEXT NOT NULL UNIQUE,
    tipo_1 TEXT NOT NULL,
    tipo_2 TEXT,
nivel INTEGER DEFAULT 50 CHECK (nivel BETWEEN 0 AND 100)
    hp_base INTEGER NOT NULL CHECK (hp_base > 0),
    ataque_base INTEGER NOT NULL CHECK (ataque_base > 0),
    defensa_base INTEGER NOT NULL CHECK (defensa_base > 0),
    ataque_especial_base INTEGER NOT NULL CHECK (ataque_especial_base > 0),
    defensa_especial_base INTEGER NOT NULL CHECK (defensa_especial_base >
0),
    velocidad_base INTEGER NOT NULL CHECK (velocidad_base > 0),
    -- Evitar que tipo_1 y tipo_2 sean iguales
    CHECK (tipo_2 IS NULL OR tipo_1 <> tipo_2)
);

CREATE TABLE equipo_pokemon(
    id_equipo_pokemon INTEGER PRIMARY KEY,
    id_equipo INTEGER NOT NULL,
    id_pokemon INTEGER NOT NULL,
    id_naturaleza INTEGER,
    id_habilidad INTEGER,
    --EVS, IVS
    ev_hp INTEGER DEFAULT 0 CHECK(ev_hp BETWEEN 0 AND 252),
    ev_ataque INTEGER DEFAULT 0 CHECK(ev_ataque BETWEEN 0 AND 252),
    ev_ataque_especial INTEGER DEFAULT 0 CHECK( ev_ataque_especial BETWEEN
0 AND 252),
    ev_defensa INTEGER DEFAULT 0 CHECK(ev_defensa BETWEEN 0 AND 252),
    ev_defensa_especial INTEGER DEFAULT 0 CHECK(ev_defensa_especial BETWEEN
```

```
0 AND 252),
    ev_velocidad INTEGER DEFAULT 0 CHECK(ev_velocidad BETWEEN 0 AND 252),

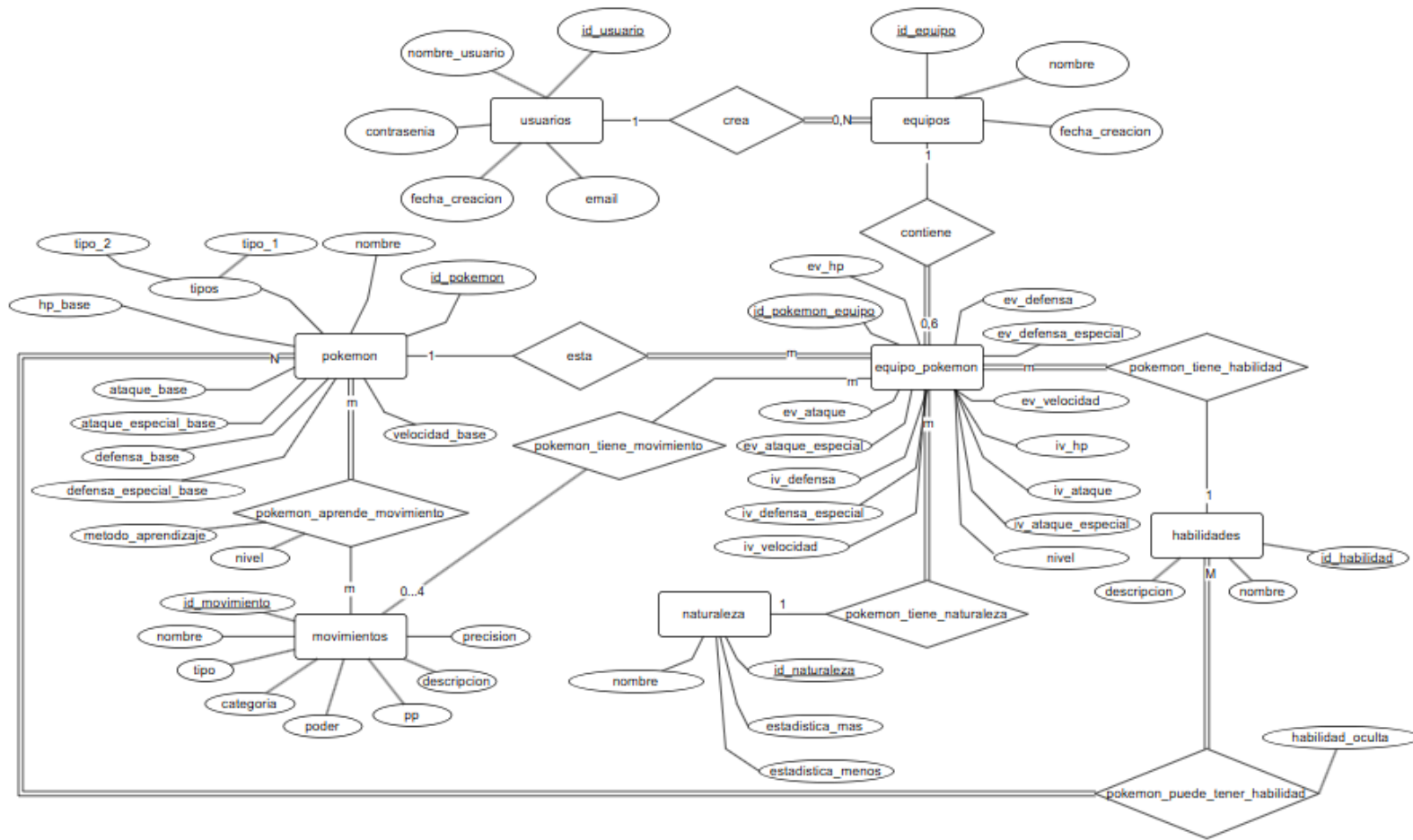
    iv_hp INTEGER DEFAULT 31 CHECK(iv_hp BETWEEN 0 AND 31),
    iv_ataque INTEGER DEFAULT 31 CHECK(iv_ataque BETWEEN 0 AND 31),
    iv_ataque_especial INTEGER DEFAULT 31 CHECK(iv_ataque_especial BETWEEN
0 AND 31),
    iv_defensa INTEGER DEFAULT 31 CHECK(iv_defensa BETWEEN 0 AND 31),
    iv_defensa_especial INTEGER DEFAULT 31 CHECK(iv_defensa_especial
BETWEEN 0 AND 31),
    iv_velocidad INTEGER DEFAULT 31 CHECK(iv_velocidad BETWEEN 0 AND 31),
    --restricciones
    CHECK((ev_hp + ev_ataque + ev_ataque_especial + ev_defensa +
ev_defensa_especial + ev_velocidad)<=510),
    FOREIGN KEY (id_equipo) REFERENCES equipos(id_equipo) ON DELETE
CASCADE,
    FOREIGN KEY (id_pokemon) REFERENCES pokemon(id_pokemon) ON DELETE
CASCADE,
    FOREIGN KEY (id_naturaleza) REFERENCES naturaleza(id_naturaleza),
    FOREIGN KEY (id_habilidad) REFERENCES habilidades(id_habilidad)

);
CREATE TABLE habilidades (
    id_habilidad INTEGER PRIMARY KEY,
    nombre TEXT NOT NULL,
    descripcion TEXT
);
CREATE TABLE movimiento (
    id_movimiento INTEGER PRIMARY KEY,
    nombre TEXT NOT NULL,
    tipo TEXT NOT NULL,
    categoria TEXT NOT NULL,
    poder INTEGER,
    pp INTEGER NOT NULL CHECK (pp > 0),
    descripcion TEXT,
precision TEXT
);

CREATE TABLE pokemon_puede_tener_habilidad (
    id_pokemon INTEGER NOT NULL,
    id_habilidad INTEGER NOT NULL,
    habilidad_oculta BOOLEAN DEFAULT FALSE,
```

```
PRIMARY KEY (id_pokemon, id_habilidad),
FOREIGN KEY (id_pokemon) REFERENCES pokemon(id_pokemon) ON DELETE
CASCADE,
FOREIGN KEY (id_habilidad) REFERENCES habilidades(id_habilidad) ON
DELETE CASCADE
);
CREATE TABLE pokemon_tiene_movimiento (
    id_pokemon_equipo INTEGER NOT NULL,
    slot INTEGER NOT NULL CHECK (slot BETWEEN 1 AND 4),
    id_movimiento INTEGER NOT NULL,
    PRIMARY KEY (id_pokemon_equipo, slot),          -- Un slot por Pokémon
    UNIQUE (id_pokemon_equipo, id_movimiento),      -- No repetir movimiento
    FOREIGN KEY (id_pokemon_equipo) REFERENCES
equipo_pokemon(id_pokemon_equipo) ON DELETE CASCADE,
    FOREIGN KEY (id_movimiento) REFERENCES movimiento(id_movimiento)
);
CREATE TABLE pokemon_aprende_movimiento (
    id_pokemon INTEGER NOT NULL,
    id_movimiento INTEGER NOT NULL,
    nivel INTEGER,
    metodo_aprendizaje TEXT,
    PRIMARY KEY (id_pokemon, id_movimiento),
    FOREIGN KEY (id_pokemon) REFERENCES pokemon(id_pokemon) ON DELETE
CASCADE,
    FOREIGN KEY (id_movimiento) REFERENCES movimiento(id_movimiento)
);
```

Figura N.º 2



Contrato API

Especificaremos los endpoint utilizados en la aplicacion:

```
export const ENDPOINTS = {  
  
  // --- USUARIO / AUTH ---  
  
  AUTH: {  
  
    LOGIN: '/auth/login',  
  
    REGISTER: '/auth/register',  
  
    PROTECTED: '/auth/protected', // Requiere Token  
  
  },  
  
  
  // --- USUARIO / PERFIL ---  
  
  USUARIOS: {  
  
    UPDATE_USERNAME: '/auth/username',  
  
  },  
  
}
```

```
    UPDATE_EMAIL: '/auth/email',

    UPDATE_PASSWORD: '/auth/password',

  },

  // --- ENCICLOPEDIA ---

  ENCICLOPEDIA: {

    POKEMONS: '/enciclopedia/pokemon', // Sirve para lista, búsqueda y filtros

    POKEMON_DETAIL: (id) => `/enciclopedia/pokemon/${id}`,

    POKEMON_MOVIMIENTOS: (id) => `/enciclopedia/pokemon/${id}/movimientos`,

    MOVIMIENTOS: '/enciclopedia/movimientos',

    NATURALEZAS: '/enciclopedia/naturalezas',

    HABILIDADES: '/enciclopedia/habilidades',

  },

  // --- EQUIPOS ---

  EQUIPOS: {

    POR_USUARIO: (id_usuario) => `/usuarios/${id_usuario}/equipos`,

    BASE: '/equipos', // Para POST (crear)

    DETALLE: (id) => `/equipos/${id}`, // Para GET, PUT y DELETE

  },

};
```

Pantallas de la Aplicación



Ruta: `src>features>home>screen>`[InicioScreen.js](#)

Pantalla de bienvenida. Es lo primero que ve el usuario después de autenticarse.

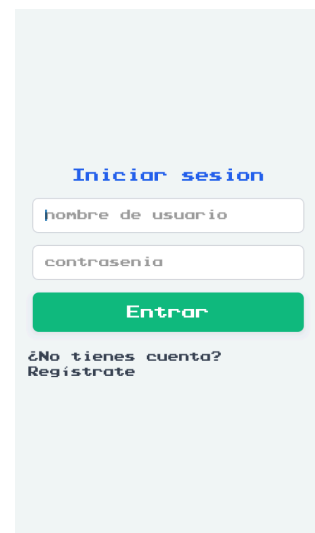
`src/features/auth/screens/`[LoginScreen.js](#)

Pantalla de inicio de sesión. El usuario ingresa su nombre de usuario y contraseña. Este puede enviar formulario de login el cual llama a un hook `useLogin`.

Además, podremos navegar a la pantalla de registro.

Endpoint consumido: `POST /auth/login`

Autentica al usuario y devuelve un token `jwt`.



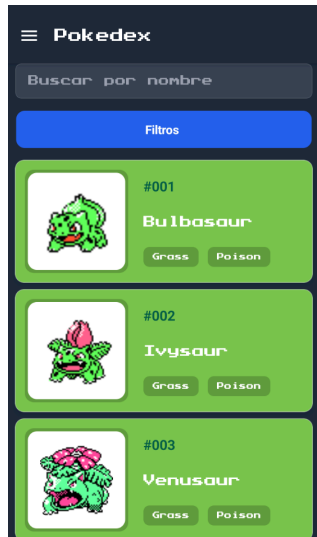
`src/features/auth/screens/`[RegisterScreen.js](#)

Pantalla de registro de nuevo usuario. El usuario completa el formulario con sus datos. Este utiliza un hook llamado `useRegister`.

Podremos volver a la pantalla de registro una vez completado el formulario o no.

Endpoint consumido: `POST /auth/register`

Este crea un nuevo usuario en la base de datos.



`src/features/pokedex/screens/`[`PokedexScreen.js`](#)

Pantalla principal de la enciclopedia pokemon. Lista todos los pokemon con busqueda, filtro por tipo y paginacion. Tambien este se reutiliza como selector de pokemon para el team builder.

Las acciones disponibles dentro de esta pantalla son:

- buscar pokemon por nombre.
- Filtrar por tipo 1 y 2.
- Scroll infinito → carga siguiente pagina.
- Presionar la CARD:
 - Modo normal: abre PokemonDetailModal con detalle del pokémon seleccionado.
 - Modo selección (`isSelecting=true`) →regresa el pokemon al team builder.
- Desde la pantalla de teamBuilder por Navigate pasamos el atributo y el slot.

```
navigation.navigate("Pokedex", {
  isSelecting: true,    // ← bande
  slotIndex: 2,        // ← qué s
}).
```

Desde la pokédex Screen esto se reinicia para evitar bugs.

Endpoint consumido:

- GET /enciclopedia/pokemon
 - params: search, type_1, type_2, page.
- GET /enciclopedia/pokemon/:id
 - Detalle de un pokemon especifico.
- GET /enciclopedia/pokemon/:id/movimientos
 - Movimientos que puede aprender ese pokemon.

src/features/moves/MovimientosScreen.js

Pantalla de enciclopedia de movimientos. Lista todos los movimientos del juego con sus estadísticas.

Buscar movimiento por nombre

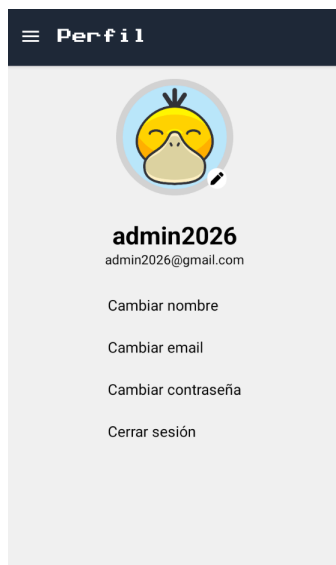
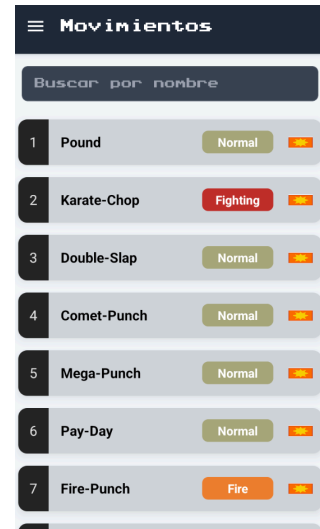
Presionar movimiento → abre MovementModal con detalle

Desde el modal → ver qué Pokémon aprenden ese movimiento

Endpoint consumibles:

GET /enciclopedia/movimientos Lista todos los movimientos

GET /enciclopedia/movimientos/:id/pokemon Pokémon que aprenden ese movimiento



src/features/profile/PerfilScreen.js

Pantalla del perfil del usuario autenticada. Muestra los datos del usuario y opciones de edición.

Podemos realizar las siguientes acciones:

- Ver datos del usuario (nombre, email)
- Abrir EditUserModal para editar nombre de usuario
- Abrir EditUserModal para editar email
- Abrir EditUserModal para cambiar contraseña
- Cerrar sesión → llama a useLogout

Endpoint consumibles:

GET /auth/me Obtiene datos del usuario autenticado (token requerido)

PUT /auth/username Actualiza el nombre de usuario

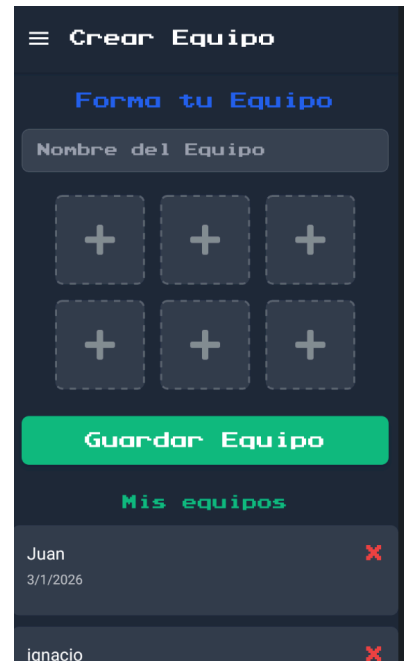
PUT /auth/email Actualiza el email

PUT /auth/password Actualiza la contraseña

src/features/team-builder/CrearEquipoScreen.js

Pantalla del Team Builder. Permite crear, visualizar y gestionar equipos Pokémon de hasta 6 miembros con sus movimientos.

- Ver lista de equipos del usuario
- Crear nuevo equipo (nombre)
- Eliminar equipo
- Presionar slot vacío → navega a PokedexScreen en modo selección
- Presionar slot con Pokémon → opciones de reemplazar/eliminar
- Asignar movimientos a un Pokémon → abre MoveSelectionModal
- Eliminar Pokémon del equipo



Endpoint consumibles:

GET /usuarios/:id/equipos Obtiene todos los equipos del usuario

POST /equipos Crea un nuevo equipo

PUT /equipos/:id Actualiza nombre del equipo

DELETE /equipos/:id Elimina un equipo